Отчёт по лабораторной работе №2

Дисциплина: Компьютерный практикум по статистическому анализу данных

Выполнил: Танрибергенов Эльдар (НПИбд-01-22)

Содержание

1 Цель работы **7**

2 Предварительные сведения **8**

3 Выполнение лабораторной работы **10**

3.1 Кортежи 10
3.2 Словари 12
3.3 Множества 14
3.4 Массивы 17
3.5 Задания для самостоятельного выполнения 28

4 Выводы **45**

Список иллюстраций

3.1 Кортеж из элементов типа String 10
3.2 Кортеж из целых чисел 10
3.3 Кортеж из элементов разных типов 10
3.4 Именованный кортеж 11
3.5 Операция length() над кортежом 11
3.6 Обращение к элементам кортежа 11
3.7 Сложение элементов кортежа 11
3.8 Обращение к элементам именованного кортежа 12
3.9 Проверка вхождения элементов в кортеж 12
3.10 Создание словаря 12
3.11 Вывод ключей словаря 12
3.12 Вывод значения элементов словаря 13
3.13 Вывод заданных в словаре пар “ключ - значение” 13
3.14 Проверка вхождения ключа в словарь 13
3.15 Добавление элемента в словарь 13
3.16 Удаление ключа и связанных с ним значений из словаря 14
3.17 Объединение словарей 14
3.18 Множество из 4 целочисленных значений 14
3.19 Множество из 11 символьных значений 15
3.20 Проверка эквивалентности двух множеств 1 15
3.21 Проверка эквивалентности двух множеств 2 15
3.22 Объединение множеств 15
3.23 Пересечение множеств 16
3.24 Разность множеств 16
3.25 Проверка вхождения элементов одного множества в другое 16
3.26 Добавление элемента в множество 16
3.27 Удаление последнего элемента множеств 17
3.28 Создание пустого массива с абстрактным типом 17
3.29 Создание пустого массива с конкретным типом 1 17
3.30 Создание пустого массива с конкретным типом 2 17
3.31 Вектор-столбец 18
3.32 Вектор-строка 18
3.33 Многомерные массивы (матрицы) 1 18
3.34 Многомерные массивы (матрицы) 2 18
3.35 Одномерный массив из 8 элементов 19
3.36 Многомерный массив 2x3 19
3.37 Трёхмерный массив 19

3.38 Массив из квадратных корней всех целых чисел от 1 до 10 20
3.39 Массив с элементами определённого вида 20
3.40 Массив квадратов элементов, если квадрат не делится на 5 или 4 . 20
3.41 Одномерный массив из пяти единиц 21
3.42 Одномерный массив из 4 нулей 21
3.43 Заполнение массива 3x2 цифрами 3.5 21
3.44 Заполнение массива посредством функции repeat 22
3.45 Преобразование одномерного массива в двумерный массив 22
3.46 Транспонирование 23
3.47 Массив 10x5 целых чисел в диапазоне 10-20 23
3.48 Выбор всех значений строки в столбце 2 24
3.49 Выбор всех значений в столбцах 2 и 5 24
3.50 Все значения строк в столбцах 2, 3 и 4 25
3.51 Значения в строках 2, 4, 6 и в столбцах 1 и 5 25
3.52 Значения в строке 1 от столбца 3 до последнего столбца 25
3.53 Сортировка по столбцам 26
3.54 Сортировка по строкам 26
3.55 Поэлементное сравнение с числом 27
3.56 Возврат индексов элементов массива 28
3.57 Задание №1 29
3.58 Задание №2 29
3.59 Задание №2 29
3.60 Задание №2 30
3.61 Задание №2 30
3.62 Задание №2 30
3.63 Задание №2 30
3.64 Задание №2 31
3.65 Задание №2 31
3.66 Задание №3.1 31
3.67 Задание №3.2 31
3.68 Задание №3.3 32
3.69 Задание №3.4 32
3.70 Задание №3.5 32
3.71 Задание №3.6 32
3.72 Задание №3.7 33
3.73 Задание №3.8 34
3.74 Задание №3.9 35
3.75 Задание №3.10 35
3.76 Задание №3.11 36
3.77 Задание №3.12 37
3.78 Задание №3.13 38
3.79 Задание №3.14 39
3.80 Задание №4 42
3.81 Задание №5 42

3.82 Задание №6.1 43
3.83 Задание №6.2 43
3.84 Задание №6.3 44

Список таблиц

1 Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

2 Предварительные сведения

Рассмотрим несколько структур данных, реализованных в Julia. Несколько функций (методов), общих для всех структур данных: – isempty() — проверяет, пуста ли структура данных; – length() — возвращает длину структуры данных; – in() — проверяет принадлежность элемента к структуре; – unique() — возвращает коллекцию уникальных элементов структуры, – reduce() — свёртывает структуру данных в соответствии с заданным бинарным оператором; – maximum() (или minimum()) — возвращает наибольший (или наименьший) результат вызова функции для каждого элемента структуры данных.

Кортеж (Tuple) — структура данных (контейнер) в виде неизменяемой индексируемой последовательности элементов какого-либо типа (элементы индексируются с единицы). Синтаксис определения кортежа: `(element1, element2, ...)`

Словарь — неупорядоченный набор связанных между собой по ключу данных. Синтаксис определения словаря: `Dict(key1 => value1, key2 => value2, ...)`

Множество, как структура данных в Julia, соответствует множеству, как математическому объекту, то есть является неупорядоченной совокупностью элементов какого-либо типа. Возможные операции над множествами: объединение, пересечение, разность; принадлежность элемента множеству. Синтаксис определения множества: `Set([itr])` где itr — набор значений, сгенерированных данным итерируемым объектом или пустое множество.

Массив — коллекция упорядоченных элементов, размещённая в многомерной сетке. Векторы и матрицы являются частными случаями массивов. Общий

синтаксис одномерных массивов: `array_name_1 = [element1, element2, ...]` `array_name_2 = [element1 element2 ...]`

Некоторые операции для работы с массивами: – length(A) — число элементов массива A; – ndims(A) — число размерностей массива A; – size(A) — кортеж размерностей массива A; – size(A, n) — размерность массива A в заданном направлении; – copy(A) — создание копии массива A; – ones(), zeros() — создать массив с единицами или нулями соответственно; – fill(value,array_name) — заполнение массива заранее определенным значением; – sort() — сортировка элементов; – collect() — вернуть массив всех элементов в коллекции или итераторе; – reshape() — изменение размера массива; – transpose() — транспонирование массива;

3 Выполнение лабораторной работы

3.1 Кортежи

Примеры кортежей:

1. Кортеж из элементов типа String:

```
[1]: favoritelang = ("Python","Julia","R")

[1]: ("Python", "Julia", "R")
```

Рис. 3.1: Кортеж из элементов типа String

2. Кортеж из целых чисел:

```
[2]: x1 = (1, 2, 3)

[2]: (1, 2, 3)
```

Рис. 3.2: Кортеж из целых чисел

3. Кортеж из элементов разных типов:

```
[3]: x2 = (1, 2.0, "tmp")

[3]: (1, 2.0, "tmp")
```

Рис. 3.3: Кортеж из элементов разных типов

4. Именованный кортеж:

```
[4]: x3 = (a=2, b=1+2)

[4]: (a = 2, b = 3)
```

Рис. 3.4: Именованный кортеж

Примеры операций над кортежами:

5. length(x2)

```
[5]: length(x2)

[5]: 3
```

Рис. 3.5: Операция length() над кортежом

6. Обратиться к элементам кортежа x2:

```
[6]: x2[1], x2[2], x2[3]

[6]: (1, 2.0, "tmp")
```

Рис. 3.6: Обращение к элементам кортежа

7. Произвести какую-либо операцию (сложение) со вторым и третьим элементами кортежа x1:

```
[7]: c = x1[2] + x1[3]

[7]: 5
```

Рис. 3.7: Сложение элементов кортежа

8. Обращение к элементам именованного кортежа x3:

```
[8]: x3.a, x3.b, x3[2]

[8]: (2, 3, 3)
```

Рис. 3.8: Обращение к элементам именованного кортежа

9. Проверка вхождения элементов tmp и 0 в кортеж x2 (два способа обращения к методу in()):

```
[9]: in("tmp", x2), 0 in x2

[9]: (true, false)
```

Рис. 3.9: Проверка вхождения элементов в кортеж

3.2 Словари

Примеры словарей и операций над ними:

1. Создать словарь с именем phonebook:

```
[10]: phonebook = Dict("Иванов И.И." => ("867-5309","333-5544"), "Бухгалтерия" => "555-2368")

[10]: Dict{String, Any} with 2 entries:
        "Бухгалтерия" => "555-2368"
        "Иванов И.И." => ("867-5309", "333-5544")
```

Рис. 3.10: Создание словаря

2. Вывести ключи словаря:

```
[11]: keys(phonebook)

[11]: KeySet for a Dict{String, Any} with 2 entries. Keys:
        "Бухгалтерия"
        "Иванов И.И."
```

Рис. 3.11: Вывод ключей словаря

3. Вывести значения элементов словаря:

```
[12]: values(phonebook)

[12]: ValueIterator for a Dict{String, Any} with 2 entries. Values:
        "555-2368"
        ("867-5309", "333-5544")
```

Рис. 3.12: Вывод значения элементов словаря

4. Вывести заданные в словаре пары “ключ - значение”:

```
[13]: pairs(phonebook)

[13]: Dict{String, Any} with 2 entries:
        "Бухгалтерия" => "555-2368"
        "Иванов И.И." => ("867-5309", "333-5544")
```

Рис. 3.13: Вывод заданных в словаре пар “ключ - значение”

5. Проверка вхождения ключа в словарь:

```
[14]: haskey(phonebook, "Иванов И.И.")

[14]: true
```

Рис. 3.14: Проверка вхождения ключа в словарь

6. Добавить элемент в словарь:

```
[15]: phonebook["Сидоров П.С."] = "555-3344"

[15]: "555-3344"
```

Рис. 3.15: Добавление элемента в словарь

7. Удалить ключ и связанные с ним значения из словаря

```
[17]: pop!(phonebook, "Иванов И.И.")

[17]: ("867-5309", "333-5544")
```

Рис. 3.16: Удаление ключа и связанных с ним значений из словаря

8. Объединение словарей (функция merge()):

```
[18]: a = Dict("foo" => 0.0, "bar" => 42.0);
      b = Dict("baz" => 17, "bar" => 13.0);
      merge(a, b), merge(b,a)

[18]: (Dict{String, Real}("bar" => 13.0, "baz" => 17, "foo" => 0.0), Dict{String, Real}("bar" => 42.0, "baz" => 17, "foo" => 0.0))
```

Рис. 3.17: Объединение словарей

3.3 Множества

Примеры множеств и операций над ними:

1. Множество из четырёх целочисленных значений:

```
[19]: A = Set([1, 3, 4, 5])

[19]: Set{Int64} with 4 elements:
        5
        4
        3
        1
```

Рис. 3.18: Множество из 4 целочисленных значений

2. Множество из 11 символьных значений:

```
[20]: B = Set("abrakadabra")

[20]: Set{Char} with 5 elements:
        'a'
        'd'
        'r'
        'k'
        'b'
```

Рис. 3.19: Множество из 11 символьных значений

3. Проверка эквивалентности двух множеств:

```
[23]: S1 = Set([1,2]);
      S2 = Set([3,4]);
      issetequal(S1,S2)

[23]: false
```

Рис. 3.20: Проверка эквивалентности двух множеств 1

```
[24]: S3 = Set([1,2,2,3,1,2,3,2,1]);
      S4 = Set([2,3,1]);
      issetequal(S3,S4)

[24]: true
```

Рис. 3.21: Проверка эквивалентности двух множеств 2

4. Объединение множеств:

```
[25]: C=union(S1,S2)

[25]: Set{Int64} with 4 elements:
        4
        2
        3
        1
```

Рис. 3.22: Объединение множеств

5. Пересечение множеств:

```
[26]: D = intersect(S1,S3)

[26]: Set{Int64} with 2 elements:
        2
        1
```

Рис. 3.23: Пересечение множеств

6. Разность множеств:

```
[27]: E = setdiff(S3,S1)

[27]: Set{Int64} with 1 element:
        3
```

Рис. 3.24: Разность множеств

7. Проверка вхождения элементов одного множества в другое:

```
[28]: issubset(S1,S4)

[28]: true
```

Рис. 3.25: Проверка вхождения элементов одного множества в другое

8. Добавление элемента в множество:

```
[30]: push!(S4, 99)

[30]: Set{Int64} with 4 elements:
        2
        99
        3
        1
```

Рис. 3.26: Добавление элемента в множество

9. Удаление последнего элемента множества:

```
[31]: pop!(S4)
[31]: 2
```

Рис. 3.27: Удаление последнего элемента множеств

3.4 Массивы

Примеры массивов:

1. Создание пустого массива с абстрактным типом:

```
[32]: empty_array_1 = []
[32]: Any[]
```

Рис. 3.28: Создание пустого массива с абстрактным типом

2. Создание пустого массива с конкретным типом:

```
[34]: empty_array_2 = (Int64)[]
[34]: Int64[]
```

Рис. 3.29: Создание пустого массива с конкретным типом 1

```
[35]: empty_array_3 = (Float64)[]
[35]: Float64[]
```

Рис. 3.30: Создание пустого массива с конкретным типом 2

3. Вектор-столбец:

```
[36]: a = [1, 2, 3]

[36]: 3-element Vector{Int64}:
       1
       2
       3
```

Рис. 3.31: Вектор-столбец

4. Вектор-строка:

```
[38]: b = [1 2 3]

[38]: 1×3 Matrix{Int64}:
       1  2  3
```

Рис. 3.32: Вектор-строка

5. Многомерные массивы (матрицы):

```
[39]: A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]

[39]: 3×3 Matrix{Int64}:
       1  4  7
       2  5  8
       3  6  9
```

Рис. 3.33: Многомерные массивы (матрицы) 1

```
[40]: B = [[1 2 3]; [4 5 6]; [7 8 9]]

[40]: 3×3 Matrix{Int64}:
       1  2  3
       4  5  6
       7  8  9
```

Рис. 3.34: Многомерные массивы (матрицы) 2

6. Одномерный массив из 8 элементов (массив 1×8) со значениями, случайно распределёнными на интервале $[0, 1)$:

```
[41]: c = rand(1,8)

[41]: 1×8 Matrix{Float64}:
 0.0500458  0.574701  0.518604  0.627241  …  0.0781264  0.683654  0.842554
```

Рис. 3.35: Одномерный массив из 8 элементов

7. Многомерный массив 2×3 (2 строки, 3 столбца) элементов со значениями, случайно распределёнными на интервале [0, 1):

```
[43]: C = rand(2,3)

[43]: 2×3 Matrix{Float64}:
 0.604089  0.143756   0.423946
 0.339329  0.0441866  0.884767
```

Рис. 3.36: Многомерный массив 2x3

8. Трёхмерный массив:

```
[44]: D = rand(4, 3, 2)

[44]: 4×3×2 Array{Float64, 3}:
[:, :, 1] =
 0.165051  0.275015  0.674567
 0.405609  0.124098  0.695134
 0.518128  0.761084  0.342936
 0.177952  0.168147  0.61943

[:, :, 2] =
 0.838742   0.46502    0.318713
 0.163425   0.839532   0.480772
 0.80575    0.375722   0.935615
 0.0732973  0.0185727  0.974309
```

Рис. 3.37: Трёхмерный массив

Примеры массивов, заданных некоторыми функциями через включение:

9. Массив из квадратных корней всех целых чисел от 1 до 10:

```
[45]: roots = [sqrt(i) for i in 1:10]
```

```
[45]: 10-element Vector{Float64}:
 1.0
 1.4142135623730951
 1.7320508075688772
 2.0
 2.23606797749979
 2.449489742783178
 2.6457513110645907
 2.8284271247461903
 3.0
 3.1622776601683795
```

Рис. 3.38: Массив из квадратных корней всех целых чисел от 1 до 10

10. Массив с элементами вида 3*x^2, где x - нечётное число от 1 до 9 (включительно)

```
[46]: ar_1 = [3*i^2 for i in 1:2:9]
```

```
[46]: 5-element Vector{Int64}:
   3
  27
  75
 147
 243
```

Рис. 3.39: Массив с элементами определённого вида

11. Массив квадратов элементов, если квадрат не делится на 5 или 4:

```
[47]: ar_2=[i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]
```

```
[47]: 4-element Vector{Int64}:
  1
  9
 49
 81
```

Рис. 3.40: Массив квадратов элементов, если квадрат не делится на 5 или 4

12. Одномерный массив из пяти единиц:

```
[48]: ones(5)

[48]: 5-element Vector{Float64}:
       1.0
       1.0
       1.0
       1.0
       1.0

[49]: ones(2,3)

[49]: 2×3 Matrix{Float64}:
       1.0  1.0  1.0
       1.0  1.0  1.0
```

Рис. 3.41: Одномерный массив из пяти единиц

13. Одномерный массив из 4 нулей:

```
[50]: zeros(4)

[50]: 4-element Vector{Float64}:
       0.0
       0.0
       0.0
       0.0
```

Рис. 3.42: Одномерный массив из 4 нулей

14. Заполнить массив 3x2 цифрами 3.5

```
[51]: fill(3.5,(3,2))

[51]: 3×2 Matrix{Float64}:
       3.5  3.5
       3.5  3.5
       3.5  3.5
```

Рис. 3.43: Заполнение массива 3x2 цифрами 3.5

15. Заполнение массива посредством функции repeat():

```
[52]: repeat([1,2],3,3)

[52]: 6×3 Matrix{Int64}:
       1  1  1
       2  2  2
       1  1  1
       2  2  2
       1  1  1
       2  2  2
```

Рис. 3.44: Заполнение массива посредством функции repeat

16. Преобразование одномерного массива из целых чисел от 1 до 12 в двумерный массив 2x6

```
[54]: a = collect(1:12)
      b = reshape(a,(2,6))

[54]: 2×6 Matrix{Int64}:
       1  3  5  7   9  11
       2  4  6  8  10  12
```

Рис. 3.45: Преобразование одномерного массива в двумерный массив

17. Транспонирование

```
[55]: b'

[55]: 6×2 adjoint(::Matrix{Int64}) with eltype Int64:
        1   2
        3   4
        5   6
        7   8
        9  10
       11  12

[56]: c = transpose(b)

[56]: 6×2 transpose(::Matrix{Int64}) with eltype Int64:
        1   2
        3   4
        5   6
        7   8
        9  10
       11  12
```

Рис. 3.46: Транспонирование

18. Массив 10x5 целых чисел в диапазоне [10, 20]:

```
[57]: ar = rand(10:20, 10, 5)

[57]: 10×5 Matrix{Int64}:
       20  15  10  19  14
       20  16  17  18  10
       19  12  15  13  13
       19  19  20  15  19
       18  20  13  17  16
       18  14  12  13  20
       16  15  13  16  15
       18  16  15  15  20
       12  19  14  11  16
       19  15  10  12  17
```

Рис. 3.47: Массив 10x5 целых чисел в диапазоне 10-20

19. Выбор всех значений строки в столбце 2:

```
[58]: ar[:, 2]

[58]: 10-element Vector{Int64}:
 15
 16
 12
 19
 20
 14
 15
 16
 19
 15
```

Рис. 3.48: Выбор всех значений строки в столбце 2

20. Выбор всех значений в столбцах 2 и 5:

```
[59]: ar[:, [2, 5]]

[59]: 10×2 Matrix{Int64}:
 15  14
 16  10
 12  13
 19  19
 20  16
 14  20
 15  15
 16  20
 19  16
 15  17
```

Рис. 3.49: Выбор всех значений в столбцах 2 и 5

21. Все значения строк в столбцах 2, 3 и 4:

```
[60]: ar[:, 2:4]

[60]: 10×3 Matrix{Int64}:
       15  10  19
       16  17  18
       12  15  13
       19  20  15
       20  13  17
       14  12  13
       15  13  16
       16  15  15
       19  14  11
       15  10  12
```

Рис. 3.50: Все значения строк в столбцах 2, 3 и 4

22. Значения в строках 2, 4, 6 и в столбцах 1 и 5:

```
[61]: ar[[2, 4, 6], [1, 5]]

[61]: 3×2 Matrix{Int64}:
       20  10
       19  19
       18  20
```

Рис. 3.51: Значения в строках 2, 4, 6 и в столбцах 1 и 5

23. Значения в строке 1 от столбца 3 до последнего столбца:

```
[62]: ar[1, 3:end]

[62]: 3-element Vector{Int64}:
       10
       19
       14
```

Рис. 3.52: Значения в строке 1 от столбца 3 до последнего столбца

24. Сортировка по столбцам:

```
[63]: sort(ar,dims=1)

[63]: 10×5 Matrix{Int64}:
 12  12  10  11  10
 16  14  10  12  13
 18  15  12  13  14
 18  15  13  13  15
 18  15  13  15  16
 19  16  14  15  16
 19  16  15  16  17
 19  19  15  17  19
 20  19  17  18  20
 20  20  20  19  20
```

Рис. 3.53: Сортировка по столбцам

25. Сортировка по строкам:

```
[64]: sort(ar,dims=2)

[64]: 10×5 Matrix{Int64}:
 10  14  15  19  20
 10  16  17  18  20
 12  13  13  15  19
 15  19  19  19  20
 13  16  17  18  20
 12  13  14  18  20
 13  15  15  16  16
 15  15  16  18  20
 11  12  14  16  19
 10  12  15  17  19
```

Рис. 3.54: Сортировка по строкам

26. Поэлементное сравнение с числом (результат - массив логических значений):

```
[65]: ar .> 14

[65]: 10×5 BitMatrix:
 1  1  0  1  0
 1  1  1  1  0
 1  0  1  0  0
 1  1  1  1  1
 1  1  0  1  1
 1  0  0  0  1
 1  1  0  1  1
 1  1  1  1  1
 0  1  0  0  1
 1  1  0  0  1
```

Рис. 3.55: Поэлементное сравнение с числом

27. Возврат индексов элементов массива, удовлетворяющих условию:

```
[66]: findall(ar .> 14)
```

```
[66]: 34-element Vector{CartesianIndex{2}}:
 CartesianIndex(1, 1)
 CartesianIndex(2, 1)
 CartesianIndex(3, 1)
 CartesianIndex(4, 1)
 CartesianIndex(5, 1)
 CartesianIndex(6, 1)
 CartesianIndex(7, 1)
 CartesianIndex(8, 1)
 CartesianIndex(10, 1)
 CartesianIndex(1, 2)
 CartesianIndex(2, 2)
 CartesianIndex(4, 2)
 CartesianIndex(5, 2)
 ⋮
 CartesianIndex(2, 4)
 CartesianIndex(4, 4)
 CartesianIndex(5, 4)
 CartesianIndex(7, 4)
 CartesianIndex(8, 4)
 CartesianIndex(4, 5)
 CartesianIndex(5, 5)
 CartesianIndex(6, 5)
 CartesianIndex(7, 5)
 CartesianIndex(8, 5)
 CartesianIndex(9, 5)
 CartesianIndex(10, 5)
```

Рис. 3.56: Возврат индексов элементов массива

3.5 Задания для самостоятельного выполнения

1.

```
[67]: A = Set([0, 3, 4, 9])
      B = Set([1, 3, 4, 7])
      C = Set([0, 1, 2, 4, 7, 8, 9])

      P = union(intersect(A,B), intersect(A,B), intersect(A,C), intersect(B,C))
```

```
[67]: Set{Int64} with 6 elements:
        0
        4
        7
        9
        3
        1
```

Рис. 3.57: Задание №1

2.

```
•[69]: S1 = Set(["red", 3.2, false, 5])
       S2 = Set(['C', 14, 1.4])
       S3 = union(S1, S2)
```

```
[69]: Set{Any} with 7 elements:
        false
        5
        1.4
        3.2
        14
        "red"
        'C'
```

Рис. 3.58: Задание №2

```
[70]: S1 = Set(["red", 3.2, false, 5])
      S2 = Set(['C', 14, 1.4])
      issetequal(S1,S2)
```

```
[70]: false
```

Рис. 3.59: Задание №2

```
[71]: S1 = Set(["red", 3.2, false, 5])
      S2 = Set(['C', 14, 1.4])
      intersect(S1,S2)

[71]: Set{Any}()
```

Рис. 3.60: Задание №2

```
[72]: S1 = Set(["red", 3.2, false, 5])
      S2 = Set(['C', 14, 1.4])
      setdiff(S2,S1)

[72]: Set{Any} with 3 elements:
        1.4
        14
        'C'
```

Рис. 3.61: Задание №2

```
[74]: S1 = Set(["red", 3.2, false, 5])
      S2 = Set(['C', 14, 1.4])
      issubset(S1,S2)

[74]: false
```

Рис. 3.62: Задание №2

```
[76]: push!(S2, "red")

[76]: Set{Any} with 5 elements:
        5
        1.4
        14
        "red"
        'C'
```

Рис. 3.63: Задание №2

```
[77]: pop!(S2)

[77]: 5
```

Рис. 3.64: Задание №2

```
[78]: Set{Any} with 4 elements:
        1.4
        14
        "red"
        'C'
```

Рис. 3.65: Задание №2

3. 3.1.

```
[79]: N = 21
arr1_1 = collect(1:N)
arr1_2 = [i for i in 1:N]
arr1_3 = collect(range(1, stop=N, length=N))

println(" Способ 1: ", arr1_1)
println(" Способ 2: ", arr1_2)
println(" Способ 3: ", arr1_3)

 Способ 1: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21]
 Способ 2: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21]
 Способ 3: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0, 17.0, 18.0, 19.0, 20.0, 21.0]
```

Рис. 3.66: Задание №3.1

3.2.

```
[81]: arr2_1 = collect(N:-1:1)
arr2_2 = [N-i+1 for i in 1:N]
arr2_3 = reverse(arr1_1)

println(" Способ 1: ", arr2_1)
println(" Способ 2: ", arr2_2)
println(" Способ 3: ", arr2_3)

 Способ 1: [21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
 Способ 2: [21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
 Способ 3: [21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Рис. 3.67: Задание №3.2

3.3.

```
[82]: arr3_1 = vcat(1:N-1, N:-1:1)
      arr3_2 = [i <= N ? i : 2*N-i for i in 1:2*N-1]
      arr3_3 = vcat(collect(1:N), collect(N-1:-1:1))

      println(" Способ 1: ", arr3_1)
      println(" Способ 2: ", arr3_2)
      println(" Способ 3: ", arr3_3)

 Способ 1: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 1
0, 9, 8, 7, 6, 5, 4, 3, 2, 1]
 Способ 2: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 1
0, 9, 8, 7, 6, 5, 4, 3, 2, 1]
 Способ 3: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 1
0, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Рис. 3.68: Задание №3.3

3.4.

```
[84]: tmp = [4, 6, 3]

[84]: 3-element Vector{Int64}:
       4
       6
       3
```

Рис. 3.69: Задание №3.4

3.5.

```
[86]: arr5 = fill(tmp[1], 10)
      arr5_alt = repeat([tmp[1]], 10)
      println(arr5)

      [4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
```

Рис. 3.70: Задание №3.5

3.6.

```
[87]: arr6 = repeat(tmp, 10)
      arr6_alt = vcat([tmp for _ in 1:10]...)
      println(arr6)

      [4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3]
```

Рис. 3.71: Задание №3.6

3.7.

```
[88]: arr7 = vcat(fill(tmp[1], 11), fill(tmp[2], 10), fill(tmp[3], 10))
```

```
[88]: 31-element Vector{Int64}:
 4
 4
 4
 4
 4
 4
 4
 4
 4
 4
 4
 6
 6
 ⋮
 6
 6
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
```

Рис. 3.72: Задание №3.7

3.8.

```
[89]: arr8 = vcat(fill(tmp[1], 10), fill(tmp[2], 20), fill(tmp[3], 30))
```

```
[89]: 60-element Vector{Int64}:
 4
 4
 4
 4
 4
 4
 4
 4
 4
 4
 6
 6
 6
 ⋮
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
```

Рис. 3.73: Задание №3.8

3.9.

```
[97]: import Base.count
elements = [2^tmp[1], 2^tmp[2], 2^tmp[3], 2^tmp[3], 2^tmp[3], 2^tmp[3]]
println("Массив: ", elements)
count_6 = 0
for elem in elements
    str_elem = string(elem)
    count_6 += Base.count(c -> c == '6', str_elem)
end
println("Количество цифр «6»: ", count_6)
```

```
Массив: [16, 64, 8, 8, 8, 8]
WARNING: import of Base.count into Main conflicts with an existing identifier; ignored.
Количество цифр «6»: 2
```

Рис. 3.74: Задание №3.9

3.10.

```
[106]: using Statistics
x = 3:0.1:6
y = exp.(x) .* cos.(x)
println("Вектор y: ", y)
mean_y = mean(y)
println("Среднее значение y: ", mean_y)
```

```
Вектор y: [-19.884530844146987, -22.178753389342127, -24.490696732801293, -26.77318244299338, -28.969237768093574, -31.0111864393
74516, -32.819774760338504, -34.30336011037369, -35.35719361853035, -35.86283371230767, -35.68773248011913, -34.68504225166807, -
32.693695428321746, -29.538816297262983, -25.032529229039966, -18.975233154958957, -11.157417389647478, -1.3620985182057503, 10.6
32038010191998, 25.046704998273004, 42.09920106253839, 61.99663027669454, 84.92906736250268, 111.0615860420258, 140.525075052787
5, 173.40577640857734, 209.73349424783467, 249.46844055885668, 292.4867067371223, 338.5643778585117, 387.36034029093076]
Среднее значение y: 53.11374594642971
```

Рис. 3.75: Задание №3.10

3.11.

```
[107]: x = 0.1
y = 0.2
i = 3:3:36
j = 1:3:34

x_i = x .^ i
y_j = y .^ j
vector = zip(x_i, y_j)

for (xi, yj) in vector
    println((xi, yj))
end
```

```
(0.001000000000000002, 0.2)
(1.000000000000004e-6, 0.001600000000000003)
(1.000000000000005e-9, 1.280000000000006e-5)
(1.000000000000008e-12, 1.024000000000006e-7)
(1.000000000000009e-15, 8.192000000000005e-10)
(1.000000000000008e-18, 6.553600000000055e-12)
(1.000000000000012e-21, 5.242880000000056e-14)
(1.000000000000012e-24, 4.194304000000005e-16)
(1.000000000000015e-27, 3.355443200000044e-18)
(1.000000000000017e-30, 2.684354560000004e-20)
(1.000000000000018e-33, 2.147483648000004e-22)
(1.000000000000002e-36, 1.717986918400035e-24)
```

Рис. 3.76: Задание №3.11

3.12.

```
[108]: M = 25
       vector_2i_over_i = [2^i / i for i in 1:M]
```

```
[108]: 25-element Vector{Float64}:
            2.0
            2.0
            2.6666666666666665
            4.0
            6.4
           10.666666666666666
           18.285714285714285
           32.0
           56.888888888888886
          102.4
          186.1818181818182
          341.3333333333333
          630.1538461538462
         1170.2857142857142
         2184.5333333333333
         4096.0
         7710.117647058823
        14563.555555555555
        27594.105263157893
        52428.8
        99864.38095238095
       190650.18181818182
       364722.0869565217
       699050.6666666666
            1.34217728e6
```

Рис. 3.77: Задание №3.12

3.13.

```
[109]: N = 30
       vector_fn = ["fn$i" for i in 1:N]
```

```
[109]: 30-element Vector{String}:
        "fn1"
        "fn2"
        "fn3"
        "fn4"
        "fn5"
        "fn6"
        "fn7"
        "fn8"
        "fn9"
        "fn10"
        "fn11"
        "fn12"
        "fn13"
        ⋮
        "fn19"
        "fn20"
        "fn21"
        "fn22"
        "fn23"
        "fn24"
        "fn25"
        "fn26"
        "fn27"
        "fn28"
        "fn29"
        "fn30"
```

Рис. 3.78: Задание №3.13

3.14.

```
[117]: import Pkg; Pkg.add("SpecialFunctions")
using Random
using SpecialFunctions

n = 250
Random.seed!(123)
x = rand(0:999, n)
y = rand(0:999, n)

vector_1 = y[2:end] .- x[1:end-1]
vector_2 = x[1:end-2] .+ 2 .* x[2:end-1] .- x[3:end]
vector_3 = sin.(y[1:end-1]) ./ cos.(x[2:end])
vector_3[abs.(cos.(x[2:end])) .< 1e-10] .= NaN
sum_result = sum(exp.(-x[2:end]) ./ (x[1:end-1] .+ 10))
y_gt_600 = y[y .> 600]
indices_y_gt_600 = findall(y .> 600)
x_corresponding = x[indices_y_gt_600]
x_mean = mean(x)
vector_sqrt_diff = sqrt.(abs.(x .- x_mean))
max_y = maximum(y)
count_within_200 = sum(abs.(y .- max_y) .<= 200)
even_count = sum(iseven.(x))
odd_count = sum(.!(iseven.(x)))
count_multiple_of_7 = sum(x .% 7 .== 0)
x_sorted_by_y = x[sortperm(y)]
top_10_x = sort(x, rev=true)[1:10]
unique_x = unique(x)
```

Рис. 3.79: Задание №3.14

```
Вектор 1 (y_i - x_(i-1)), длина: 249
[-211, -124, -794, 460, 228, 521, 626, -11, 187, 328, 298, -582, 807, -200, 177, 803, -549, -320, -499, 528, 432, -304, -800, 50
4, -234, 69, 176, -205, 77, -340, 236, 650, -515, 236, 831, 613, 595, 682, 685, -94, 382, -540, 217, -68, -74, -626, 47, 258, 48
9, -496, 138, 417, 81, 337, -528, 34, -368, -65, 30, 152, 560, 649, -224, -439, 66, -732, 231, 185, 16, -297, 643, -274, -663, -5
7, -527, 242, 54, 241, 118, -158, -547, 194, 479, 91, 471, -393, 72, 386, 111, 382, -508, -480, -578, 340, 689, 361, 76, 92, -76,
-449, 297, -1, 31, 260, -400, -864, -114, -782, 297, 175, 307, 570, -678, -529, -277, 824, -354, 127, 811, 806, -536, 56, -83, -6
20, 231, 339, 176, -221, 566, 509, 21, 109, -514, -49, 645, -9, -149, -73, 108, 176, 585, 525, 306, -237, 634, 90, 476, -709, -21
0, -975, 424, -168, 311, -172, 83, -20, -655, 63, -15, -529, 552, 29, 673, -38, -389, 415, 419, 215, 41, -127, 317, -853, 392, 34
8, -144, 131, -23, 308, 69, -614, -72, -129, 757, -113, -343, -543, -263, 387, -119, -461, 539, 762, 719, -235, -195, 268, -474,
-93, 202, 164, -226, -581, -538, 230, -97, 810, 335, 37, 88, -288, 368, 548, -2, 392, 347, -168, -111, -336, 26, -286, 7, -202, 4
30, -223, -443, 944, 299, 17, 148, -350, 58, 281, -441, 124, 203, 73, 325, -103, 776, 305, 286, -168, -29, 610, -132, -104, 576,
767, 531]

Вектор 2 (x_i + 2x_(i+1) - x_(i+2)), длина: 248
[803, 2176, 745, 850, 1261, -455, 1330, 1766, 708, 543, 2158, 451, 608, 1087, -154, 969, 1202, 1606, 978, -136, 1362, 2509, 968,
863, 554, -6, 251, 526, 1684, 1033, -196, 941, 1082, 349, 143, 958, 350, -752, 1513, 917, 1552, 650, 1863, 502, 1202, 1897, 2225,
351, 468, 1982, 397, 1564, 932, 807, 1810, 1637, 1218, 2034, 1276, 1093, 243, 803, 1247, 667, 1533, 1179, 1644, 1109, 2022, -14,
1095, 1817, 1636, 1743, 1629, 1483, -99, 616, 1059, 2539, 1184, 291, 253, -407, 625, 1671, 642, 1204, 583, 852, 1834, 2101, 667,
202, -210, 750, 2349, 281, 1134, 1882, 87, 1056, 369, 489, 2225, 553, 1510, 1705, 1467, 816, 38, 1179, 1409, 2118, 279, 910, 404,
93, -816, 1380, 1596, 1103, 1308, 2124, 295, 401, 2452, 981, -476, 539, 1058, 1115, 2124, 965, -334, 1659, 1198, 1255, 474, 371,
-60, 957, 1445, -151, 1043, -145, 1311, 1638, 2315, 1276, 1130, 1249, 1617, 984, 734, 1255, 1375, 1926, 2386, 443, 1231, -76, 160
3, 1723, 583, -162, 175, 1309, 876, -108, 2076, 880, 468, -12, 961, 1487, 1345, -422, 1200, 1044, 2125, 179, 1436, 1495, 1578, 25
54, 639, 762, 1861, 994, 222, -682, 982, 1552, 1078, 1132, 1151, 537, -141, 996, 1641, 1818, 871, 1749, 688, 393, -162, 360, 169
5, 368, -96, 1283, 1487, 24, 1450, 952, 1398, 1565, 2271, 340, 1895, 105, 1461, 1876, 460, 73, -454, 995, 1723, 1882, 1338, 1648,
1739, 1289, 735, 925, 490, 240, 419, -26, 746, 1108, 401, -300, 1960, 799, -239, -28]
```

```
Вектор 3 (sin(y_i)/cos(x_(i+1))), длина: 249
[8.571950432842645, -1.4219100205712436, -2.787213484786365, -1.0486185570049187, 0.33723601025409156, -0.831871621143209, 0.0619
17079090797966, 2.0284704845640342, -1.037142055647466, -2.0282102658162904, 1.059104212102359, -1.1770474388677485, 0.4524425273
746704, 1.2501873561202559, 8.800551037709928, 1.1639246958902556, 0.6570124610178476, -2.259980526809403, 1.0445197117210927, 1.
1136325064263977, 0.1022652276691859, -0.969454012819312, 8.3089390799393, 1.002705644119295, 0.46361462822998206, 0.997094068231
8291, -2.1831726023591127, -0.2823506385860425, -0.9361793602722307, 0.49567849323938284, -1.6759909858066744, -0.94124333660154
4, 0.9881766345549338, 0.5456059355310381, -0.36244719938470815, 0.10633662538475495, 0.2799277868136135, -0.7580784365190364, -
0.5554273945454802, -0.1480607747609249, 0.17168782260619725, 0.636716631214687, 14.995900542147977, -0.23806162104942533, -0.472
17982998780195, -0.9997213830929919, 0.9939247937356619, 0.07904325727188037, 0.4367865559464581, 24.622265870947718, -0.10613698
742575259, 1.0435314576937111, 1.8964437825931963, -0.38879261853477465, -1.0286983864533814, 0.5175752794302207, 1.1380339242298
192, 12.805551032407775, -1.4756325883859256, 0.10253402795256981, -1.060622226890729, -0.14583972189384864, -0.1113461941540640
8, 0.4841829788707354, -1.6073122270519884, 1.3713994639820009, -2.4551204926788475, -0.6341128187639588, -0.9998770587063976, 2.
6096340453962106, -2.0116944550291183, 2.060918419889414, 1.0023074653702004, -1.2904447470020843, 0.4276323862236639, 0.07892910
660777788, -0.7140897651708913, -0.5076276079943726, 0.8641598067314034, 0.9323165119727005, -0.7037152351323631, 0.1463305446032
8315, -0.7101179504204131, -6.58619017508418, 1.061390047203056, 1.223546939241329, 0.7570787630411053, -0.8393039949345528, 0.66
36146030003196, -0.009281428628450546, 0.9613101442366015, 1.395073846658312, -0.8217569705054559, -3.458869201587474, -0.1917502
5127584777, -0.7912987924864966, 0.9943319541652391, -0.25657453331112196, -1.9025006116056087, 4.996254156980909, -1.06563859929
18054, -0.07986695250184633, -0.5345826857033369, 0.49988157401970906, 0.5701955669644494, -1.0987854430290191, 0.593014211579352
4, -0.3943914755056936, 0.8023571598855332, -1.2173361315927478, 0.4599004000956453, -0.22546812593210622, -1.0327876848303101,
0.05823495979637321, -1.7310082875397046, -1.063096279495371, -0.4925679984223596, -0.995177635296152, -2.049907513665794, -0.478
3018615791639, 3.2590787447983236, 1.0226896922211273, -0.8211767191658024, -1.1750801345813977, 1.0934414496582738, 1.5693100679
837249, 1.9214946908543464, -0.9517622895469422, -1.5363494677303273, -0.8193779371107326, -0.9242902205716693, 2.963604048596782
7, 1.743924415473028, -0.1659292866927082, 0.5324897835244844, 10.588612447304175, 0.22797784019680004, 0.30529897840769515, 8.3
76057434193463, 0.9710498903741354, -15.067338756676273, 0.9965615999764564, -1.7632664557947868, 0.97200263686015, 0.44005476494
13888, -1.5637420454211588, -0.4309812844041838, 0.47734347344202016, -1.0021538153949168, 0.7484400905181752, -8.38636577369533
7, -0.6162200611026935, -0.7030813656910639, 1.0592600197843984, 0.4943688721768064, 0.5879121432331238, -8.868556728536289, 1.45
56768212618785, 1.032127673947158, 2.988997883548277, -1.0753402040490978, -0.8378699949017806, -1.9696789525674, 6.1917592755448
52, -0.8290896616925546, -5.532756780760018, 49.23779756757956, -0.26527307691660024, 1.751667631834491, 0.31507410320959806, -0.
34823339654966395, -0.1833608178315939, -1.1931699638003046, 0.602506886355148, 1.2944159895678817, 1.0817686583529442, -1.087540
8363155792, -1.7689117664936533, 0.33040998472447203, 0.7814260525261915, 4.577513205017124, 0.4599936714556408, 1.03379083323624
9, -0.9483556561329688, 0.3941043013364692, -1.1465215963711684, 5.036986579061827e-5, -3.23582038216653, 7.298034758124014, -1.0
637535839704193, -1.4515198914403973, -1.0086488211508524, 0.28582862774413564, 0.23336327438971421, -0.16087557792955237, 0.7089
822466481399, -2.9996620630885316, -0.7479385452022584, -0.018720229001316164, -0.42774122005358634, -1.1880523733346364, -1.1606
970889651405, 1.5764307556855102, -1.0385538212647762, -1.470179357070193, 2.896026390306634, -1.2664978068736443, -1.03107344099
32438, 3.940679729764019, 1.4935027639442735, 0.29832843815717836, 1.0031224271144892, 1.2673606266016848, -1.6258080629106757, -
1.3991797935258588, 0.9700148584105595, -0.9283692930721378, 0.63468505003725, 1.4195656634843825, 1.187649745474156, 0.808457120
6929761, -1.0594008310234273, 0.8624690949677388, 0.20246518865491517, -0.16623292923140878, 0.024243448477230152, -0.83919487443
81056, 1.034407691020896, 1.0596886571728477, 0.0570419563745474, 1.8660929370810269, 3.8288326436329165, 0.8973241031839134, 0.7
893790851001816, -0.6757743700260231, -0.023221628672191436, 1.5734326890202528, -1.811290872243521, 2.0524161660963367, -1.90462
23074189705, -2.1390443621166106, -0.9607237302792472, -0.4280423819612119, -0.8489412571423979, -0.7816361216795618, -2.14827216
04187494, -0.5657703750795751, -0.4144595582360843, 0.08879014793338226]

Результат суммы: 0.00017065210565897441
Элементы y > 600: [650, 753, 911, 670, 922, 767, 655, 824, 847, 905, 823, 708, 679, 738, 740, 952, 817, 981, 700, 757, 820, 769,
807, 911, 990, 947, 640, 939, 712, 954, 630, 908, 996, 845, 810, 988, 714, 658, 738, 882, 756, 745, 681, 732, 715, 838, 795, 960,
867, 628, 950, 719, 846, 829, 877, 852, 670, 975, 799, 711, 601, 697, 766, 822, 770, 802, 690, 913, 640, 675, 609, 604, 895, 956,
853, 944, 689, 642, 805, 915, 650, 627, 812, 694, 743, 907, 880, 655, 709, 804, 751, 756, 655, 607, 894, 853, 727, 872, 692, 804,
687, 713, 978, 999, 902, 945, 808, 666, 695, 928, 736, 766, 842, 922]
Индексы элементов y > 600: [5, 6, 7, 8, 9, 10, 11, 12, 14, 17, 21, 22, 23, 25, 33, 36, 37, 38, 39, 40, 41, 42, 45, 48, 49, 52, 5
3, 54, 55, 57, 59, 60, 62, 63, 68, 69, 72, 73, 75, 77, 80, 88, 89, 90, 91, 96, 98, 99, 102, 104, 110, 111, 113, 117, 120, 121, 12
3, 126, 128, 129, 130, 133, 135, 136, 138, 142, 143, 144, 146, 147, 148, 150, 152, 154, 159, 160, 162, 163, 164, 165, 170, 175, 1
77, 179, 183, 184, 185, 188, 192, 193, 194, 197, 205, 206, 207, 213, 214, 215, 217, 220, 223, 225, 227, 230, 232, 233, 235, 236,
238, 240, 245, 247, 249, 250]
Значения x на тех же позициях: [525, 390, 44, 933, 580, 327, 526, 836, 465, 720, 276, 983, 880, 380, 581, 204, 386, 18, 72, 914,
387, 771, 264, 732, 103, 223, 858, 375, 676, 706, 878, 417, 196, 585, 803, 541, 932, 840, 794, 445, 707, 295, 621, 333, 704, 180,
868, 106, 57, 195, 544, 274, 788, 456, 46, 974, 606, 87, 932, 35, 21, 671, 177, 204, 383, 165, 607, 422, 585, 133, 996, 986, 652,
693, 959, 782, 613, 132, 953, 435, 275, 195, 596, 23, 150, 993, 700, 180, 42, 32, 788, 524, 704, 84, 184, 729, 480, 202, 472, 84
3, 76, 487, 95, 858, 664, 834, 463, 321, 136, 200, 222, 2, 391, 885]
```

```
Вектор sqrt(|x − x̄|): [5.741428393701344, 9.897676495016393, 20.049039877260956, 17.263719182146122, 6.079802628375365, 9.9013130
44238123, 21.072161730586636, 21.09416981063725, 9.589786233279655, 12.689996059889065, 6.161493325485308, 18.653793179940642, 2
1.16686089149735, 4.799583315247273, 11.226575613249127, 19.64779885890529, 15.230364407984466, 9.217591876406766, 13.26514229098
203, 13.893739597387018, 14.561455971158928, 22.24778640674168, 19.798080715059225, 15.938506830942478, 10.394036751907317, 18.89
5396264698977, 20.000899979750912, 13.230117157455561, 5.003598704932281, 14.99879995199616, 16.82367379617187, 19.95083958133090
7, 9.641784067277175, 13.30548758971275, 19.15818362997912, 16.853367615998888, 10.101287046708453, 21.680313650867692, 20.396960
55788705, 20.638895319275207, 10.051666528491682, 16.82153381829374, 10.537362098741792, 19.671400560204145, 14.967832174366468,
20.321515691502935, 19.38979112832317, 15.61934697738673, 19.62233421384928, 9.94806513850809, 17.91546817696931, 16.279926289759
423, 19.234448263467293, 10.63183897545481, 13.709996353026503, 20.783743647379794, 14.763603896068195, 14.386243429054021, 19.74
7506171666334, 8.428285709442935, 7.21359826993436, 17.089060828494937, 9.847030009094112, 8.65817532740011, 4.902652343374961, 1
8.32932077301284, 9.537504914808695, 17.7472251352148, 7.27763697912997, 16.96950205515766, 20.421459301430932, 21.0704532461928
8, 18.760703611538666, 17.5203881235548, 17.49182666275881, 12.327367926690595, 6.560182924278865, 21.000857125365144, 12.2459789
31878007, 14.797432209677462, 22.47140405048158, 18.276651772138134, 19.262294775026156, 19.77463021146034, 20.73730937224017, 1
1.487558487337507, 13.600147058028453, 13.893739597387018, 11.531001691093449, 12.45134530884113, 14.695713660792388, 20.02408549
722059, 12.647687535672281, 20.100646755763854, 18.41293023937255, 17.55095439000398, 15.197499794374073, 19.492665287230476, 19.
545741224113247, 17.634171372650318, 9.053397152450565, 20.761406503413973, 10.438582279217806, 17.118294307552958, 11.4001754372
46569, 21.306430954057042, 14.527078164586298, 21.329885137993593, 12.843831204122857, 7.480909035672068, 14.629969241252697, 14.
561455971158928, 17.319468814025445, 13.600147058028453, 15.393635048291875, 21.97807998893443, 5.660035335578746, 21.93253291345
986, 20.543514791777962, 21.024652196885445, 22.04459117334681, 11.223368478313452, 10.861123330484743, 15.32853548125195, 15.998
874960446438, 20.025883251432383, 11.617400741990439, 21.07045324619288, 21.28464235076549, 21.611015709586628, 8.06002481385758
4, 9.998199837970834, 13.526418594735267, 18.082145890352727, 17.63621274537138, 16.853367615998888, 20.759672444429366, 10.24870
7235549272, 1.0178408519999584, 19.64779885890529, 16.463171018974442, 17.973202274497442, 10.907061932527935, 8.1262537494223, 2
1.955318262325417, 9.847030009094112, 18.842398998004473, 22.53805670416152, 18.054473129947603, 22.315106990556867, 4.1274689580
90418, 12.804842833865631, 12.528527447389816, 14.316563833546093, 8.604417470113825, 6.997428098951786, 16.30840274214492, 17.37
7111382505436, 21.701705002142113, 17.14537838602578, 18.73595473948419, 11.178729802620689, 18.868916238088502, 21.5630239066787
65, 7.282581959717309, 19.698629393945154, 20.881475043684055, 11.046990540414162, 10.99836351463253, 14.595752806895574, 14.3190
78182620556, 21.470072193637357, 19.571305526203407, 14.458077327224391, 17.118294307552958, 13.891148260673054, 10.3905726502440
65, 10.101287046708453, 21.564693366704752, 19.1301855714993, 6.55469297526589, 19.594999362082152, 18.385755355709485, 22.471404
05048158, 14.55898348099894, 20.247567755165065, 20.735573298078837, 17.55095439000398, 12.286740820901205, 14.420956972406513, 1
7.833563861438353, 21.119564389447053, 21.35499941465698, 17.319468814025445, 11.745807762772214, 0.18973665961010636, 5.99699924
9624765, 9.167115140544489, 17.522442752082256, 16.094595366146986, 17.086954087841402, 17.463218489156002, 15.393635048291875,
7.939521396154809, 14.695713660792388, 20.100646755763854, 17.436628114403312, 20.71318420716622, 4.902652343374961, 11.788299283
611694, 21.564693366704752, 13.529079791323577, 15.523015171029114, 2.8347839423843224, 16.912598854108733, 19.286368242880773,
4.004497471593658, 19.07784054865749, 17.028329336725903, 18.840488316389255, 17.176612005864254, 20.02408549722059, 20.298669907
16387, 21.165160051367437, 1.0178408519999584, 21.308120517774437, 19.82513556069668, 18.35854024698042, 19.051614104846866, 19.2
34448263467293, 18.867008241902052, 13.26514229098203, 18.600107526570916, 13.998714226670963, 5.003598704932281, 12.924240790081
25, 10.864437399147736, 18.76262241798838, 18.331284734027783, 16.971623375505363, 18.842398998004473, 1.9909796583591703, 10.818
317798992595, 19.027243625917023, 16.310610043772122, 19.543899303874856, 22.046224166509784, 20.323287135697317, 9.8506852553515
29, 19.923955430586567]
Элементов y, отстоящих от max(y) не более чем на 200: 57
Чётных элементов в x: 129
Нечётных элементов в x: 121
Элементов x, кратных 7: 30
x, отсортированный по возрастанию y: [471, 799, 152, 34, 100, 404, 681, 597, 311, 84, 277, 234, 870, 809, 579, 854, 190, 105, 46
4, 790, 7, 744, 463, 673, 464, 66, 725, 131, 920, 283, 815, 653, 943, 295, 851, 573, 795, 6, 425, 58, 901, 919, 889, 88, 889, 72
5, 942, 673, 377, 170, 531, 664, 137, 40, 313, 362, 640, 864, 217, 814, 638, 586, 487, 181, 371, 695, 23, 126, 117, 570, 918, 89
8, 563, 852, 205, 229, 305, 780, 684, 151, 370, 648, 492, 149, 623, 488, 614, 90, 154, 5, 618, 872, 890, 366, 71, 645, 537, 279,
47, 133, 936, 844, 52, 778, 754, 59, 696, 414, 723, 620, 824, 553, 102, 713, 719, 121, 993, 860, 521, 627, 626, 793, 776, 193, 63
9, 436, 386, 588, 75, 276, 609, 587, 875, 102, 97, 949, 21, 986, 84, 996, 195, 195, 878, 858, 585, 132, 525, 275, 526, 180, 704,
840, 321, 933, 606, 133, 880, 621, 76, 613, 607, 472, 23, 136, 671, 72, 983, 42, 35, 676, 487, 932, 704, 274, 480, 333, 222, 380,
794, 581, 150, 295, 788, 390, 707, 524, 914, 177, 2, 327, 771, 383, 868, 932, 165, 32, 843, 953, 264, 463, 803, 596, 386, 387, 20
4, 276, 836, 456, 180, 391, 585, 788, 465, 974, 959, 729, 57, 202, 46, 700, 445, 184, 652, 664, 720, 993, 417, 44, 732, 422, 435,
580, 885, 200, 375, 782, 834, 223, 544, 204, 706, 693, 106, 87, 95, 18, 541, 103, 196, 858]
Топ-10 наибольших элементов x: [996, 993, 993, 986, 983, 974, 959, 953, 949, 943]
Уникальные элементы x: [521, 586, 890, 190, 525, 390, 44, 933, 580, 327, 526, 836, 40, 465, 362, 102, 720, 573, 664, 295, 276, 98
3, 880, 234, 380, 131, 88, 313, 463, 713, 205, 90, 581, 311, 121, 204, 386, 18, 72, 914, 387, 771, 377, 875, 264, 901, 864, 732,
103, 587, 809, 223, 858, 375, 676, 920, 706, 695, 878, 417, 436, 196, 585, 563, 464, 824, 579, 803, 541, 776, 71, 932, 840, 795,
794, 640, 445, 47, 638, 707, 993, 154, 117, 97, 58, 620, 673, 621, 333, 704, 889, 648, 84, 149, 180, 719, 868, 106, 799, 570, 57,
597, 195, 618, 942, 277, 943, 653, 544, 274, 788, 725, 5, 456, 7, 66, 46, 974, 614, 606, 723, 744, 87, 623, 35, 21, 553, 588, 67
1, 815, 177, 919, 383, 487, 217, 165, 607, 422, 6, 133, 996, 814, 986, 471, 652, 645, 693, 414, 537, 754, 790, 959, 782, 137, 61
3, 132, 953, 435, 100, 52, 366, 609, 275, 283, 949, 105, 279, 681, 596, 23, 854, 531, 872, 150, 700, 898, 918, 639, 696, 170, 42,
32, 626, 488, 524, 404, 181, 229, 780, 793, 425, 184, 59, 627, 305, 729, 480, 202, 860, 472, 852, 778, 843, 193, 76, 936, 34, 95,
151, 851, 844, 834, 684, 321, 370, 136, 152, 200, 492, 371, 126, 222, 870, 2, 75, 391, 885]
```

4.

```
[118]: squares = [i^2 for i in 1:100]
```

```
[118]: 100-element Vector{Int64}:
          1
          4
          9
         16
         25
         36
         49
         64
         81
        100
        121
        144
        169
          ⋮
       7921
       8100
       8281
       8464
       8649
       8836
       9025
       9216
       9409
       9604
       9801
      10000
```

Рис. 3.80: Задание №4

5.

```
using Primes

myprimes = primes(1000)[1:168]
eighty_ninth_prime = myprimes[89]
println("89-е наименьшее простое число: $eighty_ninth_prime")

slice_89_to_99 = myprimes[89:99]
println("Срез с 89-го по 99-й элемент: $slice_89_to_99")
```

```
89-е наименьшее простое число: 461
Срез с 89-го по 99-й элемент: [461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523]
```

Рис. 3.81: Задание №5

6. 6.1.

```
[121]: sum_result = 0

       for i in 10:100
           sum_result += i^3 + 4 * i^2
       end

       println("Результат суммы: ", sum_result)

       Результат суммы: 26852735
```

Рис. 3.82: Задание №6.1

6.2.

```
[122]: M = 25
       sum_result = 0

       for i in 1:M
           term1 = 2^i / i
           term2 = 3^i / (i^2)
           sum_result += term1 + term2
       end

       println("Результат суммы: ", sum_result)

       Результат суммы: 2.1291704368143802e9
```

Рис. 3.83: Задание №6.2

6.3.

```
[123]: result = 1.0
current_product = 1.0

for numerator in 2:2:38
    denominator = numerator + 1
    current_product *= numerator / denominator
    result += current_product
end

# Выводим результат
println("Результат выражения: ", result)
```

```
Результат выражения: 6.976346137897618
```

Рис. 3.84: Задание №6.3

4 Выводы

В результате выполнения лабораторной работы, я изучил несколько структур данных, реализованных в Julia, научился применять их и операции над ними для решения задач.